Unidad 8 Manejo de Eventos

Un evento es un objeto que representa un cambio en un componente, este es generalmente producido por el usuario al realizar alguna operación, por ejemplo, escribir en un campo de texto o hacer click sobre un botón. El método que se sigue para atender un evento es el siguiente:

1) Se relaciona a un oyente (listener) con un componente para que el componente le notifique al listener cuando ocurra un evento específico.
2) Cuando se produce el evento, el listener atiende el evento.

En este método la implementación de la interface de usuario donde se produce el evento y la implementación del oyente están en clases separadas.

El manejo de eventos se puede implementar a través de las clases abstractas e interfaces que java proporciona en el paquete java.awt.event.

Interfaces
ActionListener
AdjustmentListener
AWTEventListener
ComponentListener
ContainerListener
FocusListener
InputMethodListener
ItemListener
KeyListener
MouseListener
MouseMotionListener
TextListener
WindowListener

Clases Abstractas
ComponentAdapter
ContainerAdapter
ContainerEvent
FocusAdapter
KeyAdapter
MouseMotionAdapter
WindowAdapter

8.1 Eventos generados por un marco (Frame)

En el siguiente ejemplo, se implementó una clase anidada para el manejo de Eventos, esta extiende a la clase Window Adapter y define todos sus métodos. Para vincular a la clase que maneja los eventos con el componente que los produce (en este caso el frame), se agrega un oyente (monitor) al frame.

```
import java.awt.*;
import java.awt.event.*;
class EventosFrame{
    Frame ventana;
    void  forma(){  ventana=new Frame();
                    ventana.setTitle("Prueba de Eventos de Frame");
                    ventana.setSize(200,400);
                    ventana.setBackground(new Color(100,200,1));
                    ventana.setLayout(new FlowLayout());
                    ventana.add(new Label("Hola"));
                    ventana.pack();
                    ventana.setVisible(true);
                    ManejoDeEventosFrame manejador=new ManejoDeEventosFrame();
                    ventana.addWindowListener(manejador);
    }

class ManejoDeEventosFrame extends WindowAdapter{
          public void windowActivated( WindowEvent evento){
                              System.out.println("Ventana activada");
          }

          public void windowClosed( WindowEvent evento) {
                              System.out.println("Ventana desactivada");
          }

          public void windowClosing(WindowEvent evento){
                    System.out.println("Cerrando Ventana");
                    if (evento.getSource()==ventana){
                              System.out.println("Cerrando ventana");
                              ventana.setVisible(false);
                              ventana.dispose();
                    }
          }
          public void windowDeactivated( WindowEvent evento){
                              System.out.println("Ventana desactivada");
          }

          public void windowDeiconified (WindowEvent evento){
                              System.out.println("Ventana maximizada");
          }

          public void windowIconified( WindowEvent evento){
                              System.out.println("Ventana minimizada");
          }

          public void windowOpened( WindowEvent evento){
                              System.out.println("Ventana abierta por primera vez");
          }
 }
}
public class PruebaEventosFrame {
      public static void main(String args[]){
                              System.out.println("Creando Marco");
                              EventosFrame obj= new EventosFrame();
                              obj.forma();
      }
}
```

8.2. Eventos generados por Botones

En el ejemplo, para atender los eventos generados por componentes Button, se usa una clase anidada que implementa a la interface ActionListener y se define el método actionPerformed. Es posible que dos o más botones sean atendidos por el mismo método si a cada uno se le añade el ActionListener.

```
import java.awt.*;
import java.awt.event.*;
class EventosBotones {
  Button boton1,boton2;
  Frame v1,v2;
  ManejadorEventosBoton oyeBoton;

  void forma1(){
        v1=new Frame();
        v1.setLayout(new FlowLayout());
        boton1=new Button("Abrir Frame");
        boton2= new Button("Cancelar");
        v1.add(boton1);
        v1.add(boton2);
        oyeBoton=new ManejadorEventosBoton();
        boton1.addActionListener(oyeBoton);
        boton2.addActionListener(oyeBoton);
        v1.pack();
        v1.setVisible(true);
   }

   void forma2(){
         v2=new Frame();
         v2.setTitle("Prueba de Eventos de Boton");
         v2.setSize(200,400);
         v2.setLocation(100,100);
         v2.setBackground(new Color(100,200,1));
         v2.setLayout(new FlowLayout());
         v2.add(new Label("Hola"));
         v2.pack();
         v2.setVisible(true);
   }

  class ManejadorEventosBoton implements ActionListener{
       public void actionPerformed(ActionEvent evento){
            Object fuente=evento.getSource();
            if (fuente==boton1)  forma2();
            if (fuente==boton2){
              v2.setVisible(false);
              v2.dispose();
            }
       }
    }
}
public class PruebaEventosBoton {
   public static void main(String args[]){
         EventosBotones obj=new EventosBotones();
         obj.forma1();
   }
}
```

8.3 Eventos Generados por componentes de Texto.

Para el manejo de eventos producidos por áreas de texto y/o campos texto, se inserta una clase anidada que implemente la interface TextListener. El método getText nos permite recuperar el texto escrito en el componente.

```
import java.awt.*;
class EventosTexto{
    TextField tf1;
    TextArea  areaTexto;
    Frame v;
    String txt1,txt2;
    Label etiqueta1, etiqueta2;
    ManejaEventosTexto oyeTexto;
    ManejaEventosBoton oyeBoton;
    Button b;

class ManejaEventosTexto implements TextListener{
        public void textValueChanged(TextEvent e){
            Object fuente=e.getSource();
            if (fuente==tf1){          txt1=new String(tf1.getText() );                    }
            if (fuente==areaTexto){  txt2=new String(areaTexto.getText() );      }
        }
  }
    class ManejaEventosBoton implements ActionListener{
        public void actionPerformed(ActionEvent evento){
              Object fuente=evento.getSource();
               if(fuente==b) recuperaTexto();
         }
    }

 void forma()  {
    v=new Frame();
    v.setLayout(new FlowLayout() );
    tf1 = new TextField();
    b=new Button("Mostrar Texto");
     areaTexto = new TextArea("",5, 40,TextArea.SCROLLBARS_NONE);
    areaTexto.setBackground(Color.white);
    oyeTexto=new ManejaEventosTexto();
    oyeBoton=new ManejaEventosBoton();
    tf1.addTextListener(oyeTexto);
    areaTexto.addTextListener(oyeTexto);
    b.addActionListener(oyeBoton);
    v.add(tf1);
    v.add(areaTexto);
    v.add(b);
    v.add(etiqueta1=new Label( ) );        v.add(etiqueta2=new Label( ) );
    v.pack();
    v.setVisible(true);
  }

  void recuperaTexto() {            etiqueta1.setText(txt1);
                                     etiqueta2.setText(txt2);
  }

}

public class PruebaEventosTexto{
   public static void main(String args[]){
          EventosTexto pagina=new EventosTexto();
          pagina.forma();
    }
}
```

8.4 Aplicaciones con eventos generados por distintos componentes.

```
import java.awt.event.*;
class ConvierteDolares{
   TextField tPesos,tDolares;
   Frame v;
   ManejaEventosTexto oyeTexto;
   ManejaEventosBoton oyeBoton;
   Button b;
   class ManejaEventosTexto implements TextListener{
          public void textValueChanged(TextEvent e){
                    Object fuente=e.getSource();
                    /*if (fuente==tPesos){   System.out.println(tPesos.getText() );}*/
          }
   }

   class ManejaEventosBoton implements ActionListener{
                    public void actionPerformed(ActionEvent evento){  Object fuente=evento.getSource();
                                                                      if(fuente==b)     cambiaDolares();
                   }
   }

 void forma()  {
   v=new Frame();
   v.setLayout(new FlowLayout() );
   tPesos = new TextField();
   tDolares= new TextField("0") ;
   b=new Button("Convertir");
   oyeTexto=new ManejaEventosTexto();
   oyeBoton=new ManejaEventosBoton();
   tPesos.addTextListener(oyeTexto);
   b.addActionListener(oyeBoton);
   v.add(new Label("Pesos"));       v.add(tPesos);
   v.add(new Label("Dolares") );    v.add(tDolares);
   v.add(b);
   v.pack();
   v.setVisible(true);
  }

  void cambiaDolares() {
       double pesos=0;
        try{      pesos=new Double(tPesos.getText() ).doubleValue() ;      }
        catch(Exception exp){    tPesos.setText("");           }
        double dolares=pesos/11.45;
        System.out.println("Pesos=" +pesos);
        String cambio= new String();
        System.out.println("Dolares="+ cambio.valueOf(dolares));
         tDolares.setText(cambio.valueOf(dolares) );
  }
}
public class PruebaConvierteDolares{
   public static void main(String args[]){
         ConvierteDolares pagina=new ConvierteDolares();
         pagina.forma();
   }
}
```

8.5 Eventos generados por el componente List.

Los eventos producidos por un componente de la clase List se manejan a través de una clase anidada que implementa la interface ItemListener y define el método itemStateChanged. Los elementos seleccionados pueden recuperarse como String usando los métodos getSelectedItem y getSelectedItemes. También pueden recuperarse los índices de los elementos seleccionados con getSelectedIndex y getSelectedIndexes .

8.5.1 Componente List - Selección Simple

```
import java.awt.*;
import java.awt.event.*;

class EventosListaSimple {
   Frame v;
   List lista;
   ManejaEventosListaSimple oyeLista;

   class ManejaEventosListaSimple implements ItemListener{
        public void itemStateChanged(ItemEvent e) {
          Object fuente=e.getSource();

          if (fuente==lista){
             System.out.println(lista.getSelectedItem());

          }
         }

   }

   void forma(){
   v=new Frame();
   v.setLayout(new FlowLayout() );
   lista = new List(4,false);
   lista.add("lunes");
   lista.add("martes");;
   lista.add("miercoles");
   lista.add("jueves");
   lista.add("viernes");
   lista.add("sabado");
   lista.add("domingo");
   lista.addItemListener(oyeLista=new ManejaEventosListaSimple() );
   v.add(lista);
   v.pack();
   v.setVisible(true);
   }
}

public class PruebaEventosListaSimple{
   public static void main(String args[]){
         EventosListaSimple pagina=new EventosListaSimple();
         pagina.forma();
   }
}
```

8.5.2 Componente List - Selección Múltiple

```
import java.awt.*;
import java.awt.event.*;

class EventosListaMultiple {
   Frame v;
   List lista;
   ManejaEventosListaMultiple oyeLista;

   class ManejaEventosListaMultiple implements ItemListener{
        public void itemStateChanged(ItemEvent e) {
          Object fuente=e.getSource();

          if (fuente==lista){
          for(int i=0;i<lista.getSelectedItems().length;i++){
             System.out.println(lista.getSelectedItems()[i]);
          System.out.println();
          }
         }
       }
   }

   void forma(){
   v=new Frame();
   v.setLayout(new FlowLayout() );
   lista = new List(4,true);
   lista.add("lunes");
   lista.add("martes");;
   lista.add("miercoles");
   lista.add("jueves");
   lista.add("viernes");
   lista.add("sabado");
   lista.add("domingo");
   lista.addItemListener(oyeLista=new ManejaEventosListaMultiple() );
   v.add(lista);
   v.pack();
   v.setVisible(true);
   }
}

public class PruebaEventosListaMultiple{
   public static void main(String args[]){
         EventosListaMultiple pagina=new EventosListaMultiple();
         pagina.forma();
   }
}
```

8.6 Eventos generados por el componente Choice.

Los eventos generados por los componentes Choice pueden atenderse con la implementación de la interface ItemListener y la definición del método itemStateChanged. El método getSelectedItem regresa el elemento que haya sido seleccionado por el usuario.

```
import java.awt.*;
import java.awt.event.*;

class EventosChoice {
   Frame v;
   Choice selec;
   ManejaEventosChoice oyeChoice;

   class ManejaEventosChoice implements ItemListener{
        public void itemStateChanged(ItemEvent e) {
          Object fuente=e.getSource();
          if (fuente==selec)
             System.out.println("Se eligio la opcion" + selec.getSelectedItem() );
         }
   }

   void forma(){
   v=new Frame();
   v.setLayout(new FlowLayout() );
   selec= new Choice();
   selec.add("< 18");
   selec.add("18-30");
   selec.add("30-40");
   selec.add("> 40");
   selec.setForeground(Color.yellow);
   selec.setBackground(Color.pink);
   selec.addItemListener(oyeChoice =new ManejaEventosChoice() );
   v.add(selec);
   v.pack();
   v.setVisible(true);
   }
}

public class PruebaEventosChoice{
   public static void main(String args[]){
         EventosChoice pagina=new EventosChoice();
         pagina.forma();
   }
}
```

8.7 Eventos generados por el componente Checkbox.

En el siguiente programa, el manejo de los eventos que se provocan al hacer cambios en alguno de los componentes Checkbox se implementa usando una clase anidada que define el método itemState. La clase Checkbox proporciona el método getState(), el cual regresa el estado (true/false) del componente que provocó el evento.

```
import java.awt.*;
import java.awt.event.*;

class EventosCheckbox {
  Frame v;
  Checkbox caja1,caja2,caja3;
  ManejaEventosCheckbox oyeCheckbox;

  class ManejaEventosCheckbox implements ItemListener{
      public void itemStateChanged(ItemEvent e) {
        Object fuente=e.getSource();
        if (fuente==caja1)
           System.out.println("opcion 1= " + caja1.getState() );
        if (fuente==caja2)
           System.out.println("opcion 2=" + caja2.getState() );
        if (fuente==caja3)
           System.out.println("opcion 3=" + caja3.getState() );
       }
  }

  void forma(){
              v=new Frame();
              v.setLayout(new FlowLayout() );
              caja1= new Checkbox("Opcion 1");
              caja2= new Checkbox("Opcion 2");
              caja3= new Checkbox("Opcion 3", true);
              caja1.addItemListener(oyeCheckbox=new ManejaEventosCheckbox () );
              caja2.addItemListener(oyeCheckbox);
              caja3.addItemListener(oyeCheckbox);
              v.add(caja1);
              v.add(caja2);
              v.add(caja3);
              v.pack();
              v.setVisible(true);
  }
}

public class PruebaEventosCheckbox{
  public static void main(String args[]){
       EventosCheckbox pagina=new EventosCheckbox();
       pagina.forma();
  }
}
```

8.8 Eventos generados por un componente CheckboxGroup

Al igual que en el ejemplo anterior, se requiere implementar la interfaz ItemListener y que cada componente Checkbox este vinculado con un objeto Itemlistener. En este caso una vez que se produce el evento, el método getSelectedCheckbox regresa el objeto Checkbox que lo generó y con getLabel se obtiene la etiqueta que lo identifica.

```
import java.awt.*;
import java.awt.event.*;

class EventosCheckboxGroup {
   Frame v;
   Checkbox caja1,caja2,caja3;
   ManejaEventosCheckboxGroup oyeCheckboxGroup;
   CheckboxGroup grupo;
   class ManejaEventosCheckboxGroup implements ItemListener{
        public void itemStateChanged(ItemEvent e) {

        System.out.println("Se selecciono "+grupo.getSelectedCheckbox().getLabel() );

         }
   }

   void forma(){
   v=new Frame();
   Panel p=new Panel();
   v.setLayout(new FlowLayout() );

   grupo= new CheckboxGroup();

    Checkbox  caja1= new Checkbox("azul", grupo, true);
    Checkbox  caja2= new Checkbox("rojo", grupo, false);
    Checkbox  caja3= new Checkbox("amarillo", grupo, false);

    caja1.addItemListener(oyeCheckboxGroup=new ManejaEventosCheckboxGroup () );
    caja2.addItemListener(oyeCheckboxGroup);
    caja3.addItemListener(oyeCheckboxGroup);
    p.add(caja1);
    p.add(caja2);
    p.add(caja3);
    v.add(p);
    v.pack();
    v.setVisible(true);
   }
}

public class PruebaEventosCheckboxGroup{
   public static void main(String args[]){
         EventosCheckboxGroup pagina=new EventosCheckboxGroup();
         pagina.forma();
   }
}
```

8.9 Eventos generados por el componente MenuBar

El componente MenuBar por sí mismo no genera eventos, MenuBar trabaja en conjunto con MenuItem y es este quién si genera eventos. El manejo de eventos de MenuItem se hace implementado en una clase anidada el método actionPerformed de la interfaz ActionListener. A cada elemento MenuItem del menú de barra debe asociarsele con el oyente (listener). Cuando se produce el evento, getSource identifica el componente que fue activado y mediante una comparación se pueden ejecutar las acciones correspondientes a la opción seleccionada.

```
import java.awt.*;
import java.awt.event.*;
class EventosComponentesMenuBar {
  MenuBar principal;
  Menu quimicas,contab;
  Frame v;
  MenuItem quim1,quim2,quim3,cont1,cont2,cont3;
  ManejaEventosMenuBar oyeMenu;

  class ManejaEventosMenuBar implements ActionListener{
       public void actionPerformed(ActionEvent e) {
         Object fuente=e.getSource();
         if (fuente==quim1)  System.out.println("Se eligio Ing. en Computacion");
         if (fuente==quim2)  System.out.println("Se eligio Ing. en Electronica");
         if (fuente==quim3)  System.out.println("Se eligio Ing. Quimico");
         if (fuente==cont1)   System.out.println("Se eligio Contador Publico");
         if (fuente==cont2)   System.out.println("Se eligio LAE");
         if (fuente==cont3)   System.out.println("Se eligio Lic. Informatica");
       }
  }

  void forma(){      v=new Frame();
                        principal=new MenuBar();
                        quimicas=new Menu("FCQI");
                        contab=new Menu("FCA");
                        quim1 =new MenuItem(" Ing.en Computacion ");
                        quim2 =new MenuItem(" Ing.en Electronica ");
                        quim3 =new MenuItem(" Ing.Quimico ");
                        cont1  =new MenuItem(" Contador Publico ");
                        cont2 =new MenuItem(" LAE ",new MenuShortcut(KeyEvent.VK_L,false) );
                        cont3 =new MenuItem(" Lic. Inf");
                        Panel p=new Panel();    v.setTitle("Prueba de Eventos de  MenuBar");
                        v.setLayout(new FlowLayout() );
                        v.setMenuBar(principal);
                        principal.add(quimicas);
                        quimicas.add(quim1);    quimicas.add(quim2);    quimicas.add(quim3);
                        principal.add(contab);
                        contab.add(cont1);      contab.add(cont2);      contab.add(cont3);
                        quim1.addActionListener(oyeMenu=new ManejaEventosMenuBar() );
                        quim2.addActionListener(oyeMenu);
                        quim3.addActionListener(oyeMenu);
                        cont1.addActionListener(oyeMenu);
                        cont2.addActionListener(oyeMenu);
                        cont3.addActionListener(oyeMenu);
                        v.pack();
                        v.setVisible(true);
  }
}
 class PruebaEventosComponentesMenuBar{
  public static void main(String args[]){
        EventosComponentesMenuBar pagina=new EventosComponentesMenuBar();
        pagina.forma();
  }
}
```

8.10 Componente MenuPopUp y Eventos generados por el Mouse

La clase MenuPopUp implementa un menú el cual puede ser desplegado en forma dinámica; es decir, puede ubicarse en distintas posiciones de la pantalla. Generalmente esta ligado a un evento del mouse.

8.10.1 Constructores de la Clase MenuPopUp

Constructor	Descripción
MenuPopUp()	Crea un nuevo menú Pop-up sin etiqueta.
MenuPopUp(String)	Crea un nuevo menú Pop-up con la etiqueta.

8.10.2 Métodos de la clase MenuPopUp

Método	Descripción
add(MenuItem m);	Agrega la opción m al menú.
add(String e);	Agrega una opción con la etiqueta e al menú.
addSeparator();	Inserta una línea (separador) en el menú.

8.11 Métodos de la interface MouseEvent

Método	Descripción
short getButton();	En eventos ocasionados por oprimir o liberar un botón del mouse, regresa el botón que haya cambiado de estado.
int getScreenX();	Regresa la coordenada horizontal en la cual el evento ocurrió.
int getScreenY();	Regresa la coordenada vertical en la cual el evento ocurrió.

Las opciones que presenta un Menú Pop-Up son componentes MenuItem y se tratan de la misma manera que las opciones de un componente Menu.

En el ejemplo, el menú pop-up aparecerá cada vez que el usuario hace click con el boton derecho del mouse, por lo que es necesario implementar el manejo de eventos generados por el mouse. Esto se hace extendiendo la clase abstracta MouseAdapter y agregando el oyente para el mouse al frame. Cuando se presiona un botón del mouse se genera un objeto de la clase MouseEvent. El método getButton() regresa el botón presionado representado por alguno de los campos estáticos de MouseEvent: BUTTON1, BUTTON2 y BUTTON3.

Para desplegar el menú Pop-Up la clase encargada del evento recibe el frame donde se generó el evento y el menú que se desea mostrar. Para que el menú aparezca en la posición (x,y) donde se hizo click es necesario extraer las coordenadas del objeto MouseEvent, esto se hace con los métodos getX() y getY().

8.12 Ejemplo de las clases MenuPopUp y MouseEvent

```
import java.awt.event.*;
import java.awt.*;

class MenuPopUp{
   MenuBar principal;
   Menu quimicas,contab;
   Frame v;
   MenuItem quim1,quim2,quim3,cont1,cont2,cont3;
   ManejaEventosMenuBar oyeMenu;

    class ManejaEventosMenuBar implements ActionListener{
        public void actionPerformed(ActionEvent e) {
          Object fuente=e.getSource();
          if (fuente==quim1)       System.out.println("Se eligio Ing. en Computacion");
          if (fuente==quim2)       System.out.println("Se eligio Ing. en Electronica");
          if (fuente==quim3)       System.out.println("Se eligio Ing. Quimico");
          if (fuente==cont1)       System.out.println("Se eligio Contador Publico");
          if (fuente==cont2)       System.out.println("Se eligio LAE");
          if (fuente==cont3)       System.out.println("Se eligio Lic. Informatica");
         }
   }

class ManejaEventosMouse extends MouseAdapter{
    Frame marco;
    PopupMenu mpopup;
    ManejaEventosMouse( Frame frame,PopupMenu mP ) {
       marco = frame;
       mpopup = mP;
    }

   public void mousePressed( MouseEvent event ) {
       if( event.getButton()==MouseEvent.BUTTON1)
                    System.out.println("Se presiono el boton 1");
       if( event.getButton()==MouseEvent.BUTTON2)
                    System.out.println("Se presiono el boton 2");
       if( event.getButton()==MouseEvent.BUTTON3)
                    System.out.println("Se presiono el boton 3");
       if( event.getY() > 0  && event.getButton()==MouseEvent.BUTTON3  )
          mpopup.show( marco,event.getX(),event.getY() );
     }
  }

class ManejaEventosMenuItem implements ActionListener {
           public void actionPerformed( ActionEvent evt ) {
                    System.out.println( evt.getSource() );
                    }
}

class EventoFrame extends WindowAdapter {
           public void windowClosing( WindowEvent evt ) {
                      System.exit( 0 );
                    }
}
```

… continua ejemplo 8.12

```
public void forma() {
              principal=new MenuBar();
              quimicas=new Menu("FCQI");
              contab=new Menu("FCA");
              quim1 =new MenuItem(" Ing.en Computacion ");
              quim2 =new MenuItem(" Ing.en Electronica ");
              quim3 =new MenuItem(" Ing.Quimico ");
              cont1 =new MenuItem(" Contador Publico ");
              cont2 =new MenuItem(" LAE ",new MenuShortcut(KeyEvent.VK_L,false) );
              cont3 =new MenuItem(" Lic. Inf");
              Panel p=new Panel();
              v=new Frame("Prueba de Eventos de Menu Pop-Up");
              v.setLayout( new FlowLayout() );
              v.setMenuBar(principal);
              principal.add(quimicas);
              quimicas.add(quim1);
              quimicas.add(quim2);
              quimicas.add(quim3);
              principal.add(contab);
              contab.add(cont1);
              contab.add(cont2);
              contab.add(cont3);
              MenuItem primerElementoMenu = new MenuItem( "Primer Elemento" );
              MenuItem segundoElementoMenu= new MenuItem( "Segundo Elemento" );
              MenuItem tercerElementoMenu = new MenuItem( "Tercer Elemento" );
              primerElementoMenu.addActionListener ( new ManejaEventosMenuItem() );
              segundoElementoMenu.addActionListener( new ManejaEventosMenuItem() );
              tercerElementoMenu.addActionListener ( new ManejaEventosMenuItem() );
              PopupMenu menuPopup = new PopupMenu( "Menu Popup" );
              menuPopup.add( primerElementoMenu );
              menuPopup.add( segundoElementoMenu);
              menuPopup.add( tercerElementoMenu );
              quim1.addActionListener(oyeMenu=new ManejaEventosMenuBar() );
              quim2.addActionListener(oyeMenu);
              quim3.addActionListener(oyeMenu);
              cont1.addActionListener(oyeMenu);
              cont2.addActionListener(oyeMenu);
              cont3.addActionListener(oyeMenu);
              v.addMouseListener( new ManejaEventosMouse( v,menuPopup ) );
              v.addWindowListener( new EventoFrame() );
              v.setSize( 250,100 );
              v.add( menuPopup );
              v.pack();
             v.setVisible( true );

     }
   }

 class PruebaMenuPopUp {
          public static void main( String args[] ) {
                    MenuPopUp objeto= new MenuPopUp();
                    objeto.forma();
           }
}
```